

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

17. Definición de clases

Clases predefinidas

```
class Eq a where
    (==), (/=) :: a -> a -> Bool
    x /= y = not (x == y)
    x == y = not (x /= y)
```

- `Eq` es el nombre de la clase de tipos que estamos definiendo.
- `a` es una variable de tipo. Le damos nombre para usarla en las líneas siguientes.
- `(==), (/=) :: a -> a -> Bool` dice que, para que un tipo `a` pueda considerarse legalmente una instancia de la clase `Eq`, debe definir los operadores `==` y `/=`.
- Las otras dos líneas son definiciones por defecto para los operadores.
 - Si una instancia define solo uno de ellos, el otro ya está.
 - Conjunto mínimo de operadores necesarios: `==` o `/=`.

Definir nuevas clases

```
class Describable a where
    describir :: a -> String
    describir _ = "dato sin descripción"
```

- No es obligatorio que haya definiciones por defecto.
- A las funciones de una clase también se las llama *métodos*.

```
class (Eq a, Enum a, Bounded a) => Ciclico a where
    siguiente, anterior :: a -> a
    siguiente x
        | x == maxBound = minBound
        | otherwise      = succ x
    anterior x
        | x == minBound = maxBound
        | otherwise      = pred x
```

- Se dice que `Eq`, `Enum` y `Bounded` son *superclases* de `Ciclico`.
- Para declarar `instance Ciclico T`, antes `T` tiene que ser instancia de `Eq`, `Enum` y `Bounded`.

Declarar instancias

```
data Dia = Lun | Mar | Mie | Jue | Vie | Sab | Dom
    deriving (Show, Eq, Enum, Bounded)

instance Ciclico Dia
instance Ciclico Bool
```

- Valen todas las definiciones por defecto.
 - No se escribe el `where`.

```
data Semaforo = Rojo | Ambar | Verde
    deriving (Show, Eq, Enum, Bounded)

instance Describable Semaforo where
    describir Rojo  = "no te muevas"
    describir Ambar = "ten cuidado"
    describir Verde = "vía libre"

instance Ciclico Semaforo where
    siguiente Verde = Ambar
    siguiente Ambar = Rojo
    siguiente Rojo  = Verde
    anterior  Rojo  = Ambar
    anterior  Ambar = Verde
    anterior  Verde = Rojo

data ArbolBin a = Vacio | NodoBin a (ArbolBin a) (ArbolBin a)
    deriving Show

instance Eq a => Eq (ArbolBin a) where
    Vacio == Vacio = True
    NodoBin x iz1 dr1 == NodoBin y iz2 dr2 =
        x == y && iz1 == iz2 && dr1 == dr2
    _ == _ = False

class Sumable a where
    suma :: a -> Int

instance Sumable Int where
    suma x = x

instance (Sumable a, Sumable b) => Sumable (a, b) where
```

```

    suma (x, y) = suma x + suma y

unaSuma :: Int
unaSuma = suma ((1::Int, (2::Int, 5::Int)), (4::Int, 1::Int))  -- = 13

```

Fuera de menú

DeriveAnyClass

```

{-# LANGUAGE DeriveAnyClass #-}

data Dia = Lun | Mar | Mie | Jue | Vie | Sab | Dom
    deriving (Show, Eq, Enum, Bounded, Ciclico)

```

- `DeriveAnyClass` es una extensión de GHC.
- Solo vale para clases que tengan definiciones por defecto para todos sus métodos.

Clases de constructores de tipos

```

class Contenedor f where
    vacio    :: f a
    inserta  :: a -> f a -> f a
    aLista   :: f a -> [a]

instance Contenedor [] where
    vacio    = []
    inserta  = (:)
    aLista   = id

instance Contenedor ArbolBin where
    vacio    = Vacio
    inserta x Vacio = NodoBin x Vacio Vacio
    inserta x (NodoBin y iz dr) = NodoBin y (inserta x dr) iz
    aLista Vacio = []
    aLista (NodoBin x iz dr) = aLista iz ++ [x] ++ aLista dr

desdeLista :: Contenedor f => [a] -> f a
desdeLista []      = vacio
desdeLista (x:xs) = inserta x (desdeLista xs)

```

- En `Contenedor f`, `f` es un constructor de tipos: la instancia es `Contenedor ArbolBin`, no `Contenedor (ArbolBin a)`.
- `Foldable` es una clase de constructores de tipos predefinida: por eso `:t sum` muestra `(Foldable t, Num a) => t a -> a`.

Ejercicios

1. Queremos definir la igualdad en el tipo de las funciones `Int -> Int` dada por su valor en 0, es decir, en notación matemática, $f = g \iff f(0) = g(0)$. (Es una definición discutible, pero dejémoslo.) Haz de ese tipo de funciones una instancia de `Eq`.
2. Queremos hacer que esa declaración `instance` sea algo más general. Buscamos algo como lo de abajo. Añade las restricciones de clase necesarias para `a` y `b`.

```
| instance Eq (a -> b) where  
|     ...
```

3. Queremos ser aún más generales. Elegir el punto 0 para comprobar la igualdad es arbitrario. Lo único que necesitamos es que haya un punto destacado dentro del tipo. En matemáticas se suele llamar *conjunto punteado* (*pointed set*) a un conjunto acompañado de uno de sus elementos. Define la clase de tipos `Punteado` y generaliza la instancia `Eq (a -> b)`.
4. Define el tipo `Franja`, con los valores `Manana`, `Tarde` y `Noche`, y haz de él una instancia de `Ciclico`. Después, declara la instancia `Ciclico (a, b)`, de modo que los pares `(Dia, Franja)` se recorran de la forma natural: detrás de `(Lun, Noche)` va `(Mar, Manana)`, y detrás de `(Dom, Noche)`, `(Lun, Manana)`. Puede que necesites definir otras instancias por el camino.